

Project Design Draft (v0.1)

Hardware-Bound Zero-Knowledge Authentication for IoT Devices

A student design draft. Written to be readable from scratch — every term is explained in plain language the first time it appears, and every security claim is backed by a proof and a runnable demonstration (see [redesign_proof_of_concept/hardware_bound_schnorr_poc.py](#)).

Status: design + working proof-of-concept (software). Not yet on hardware. **Replaces:** the earlier “ZKP-MFA on ESP32” design, whose problems are summarised in [PAPER_VS_CODE_AUDIT.md](#).

0. TL;DR (read this first)

We want a cheap IoT device (an ESP32, ~\$5) to let a user log in with a **4-digit PIN**, such that:

1. The PIN is **never sent over the network** — not even encrypted.
2. If a hacker **steals the whole server database**, they still **cannot recover the PIN**.

The earlier version of this project tried to do this and failed at both: the device actually sent the PIN in clear text, and the PIN could be recovered from the stored data in under a millisecond. This draft fixes the root cause with one key idea:

Bind the PIN to a secret key that lives inside the chip. The login secret is not the PIN alone — it is **PIN** mixed with a 256-bit key stored in the ESP32’s tamper-resistant memory. Now stealing the server tells the attacker nothing, because they’d also need to physically extract the chip’s key.

Everything below explains *why* this works, *proves* it, and lays out *how to build it*.

1. Background — the words you need (plain-language glossary)

Term	Plain meaning
Authentication	Proving you are who you claim to be (e.g., typing your PIN).
Factor	A <i>category</i> of proof. Three kinds: knowledge (something you know — a PIN), possession (something you have — a phone or a chip), inherence (something you are — a fingerprint/face). “Multi-factor” = using two or more.
Entropy	How hard something is to guess, measured in bits. A 4-digit PIN has 10,000 options $\approx$ 13 bits — very low. A random 256-bit key has 2^{256} options — astronomically high.
Hash / KDF	A one-way blender. A hash (e.g., SHA-512) turns any input into a fixed-size scrambled output you can’t reverse. A KDF (key-derivation function) is a hash used to turn secrets into keys. We write $H(a \parallel b)$ for “hash of a joined with b”.
Elliptic curve	A special set of points you can “add” together with defined math. It gives the same security as old-style big-number crypto but with much smaller, faster keys . Think of it as the modern engine.
Ed25519 / Curve25519	A specific, fast, trusted elliptic curve. Ed25519 = using it for <i>signatures</i> / <i>proofs of a key</i> . Curve25519 (X25519) = using it for <i>key exchange</i> . Same curve, two jobs. We use the Ed25519 (proof) side.
B (base point)	A fixed public point on the curve that everyone agrees on. “Multiplying” it by a secret number gives your public key.
L (group order)	A big (253-bit) prime. All secret numbers are taken “mod L ”. You don’t need to know why — just that arithmetic wraps around L .
Public / private key	You pick a secret number x (private). Your public key is the curve point $Y = x \cdot B$. Given Y , no one can work backwards to x — <i>that’s</i> the hard problem the security rests on.
Discrete Logarithm Problem (DLP)	The “work backwards” step above (find x from $Y = x \cdot B$) is believed to be computationally impossible for big x . This is our foundation.
Zero-Knowledge Proof (ZKP)	A way to prove “ <i>I know a secret</i> ” without revealing the secret itself. Like proving you know a password by doing a challenge, without ever saying the password.
Schnorr protocol	The specific, simple ZKP we use to prove knowledge of x . (Fun fact: Ed25519 signatures are basically

Term	Plain meaning
	Schnorr proofs — so we are using the standard, battle-tested version.)
Prover / Verifier	The prover (the ESP32 device) is trying to convince the verifier (the server) that it knows the secret.
Secure element / eFuse	Special hardware inside many chips that stores a key so it can't be read out by software , even by someone who dumps the firmware. The ESP32 has this (eFuse + flash encryption). This is what makes our idea possible.
Dictionary / brute-force attack	Just trying every possible value. For a 4-digit PIN that's only 10,000 tries — instant on any laptop.
Honeypot	Here it means: does the stored data become a “jar of honey” for attackers (i.e., does stealing it hand them the secrets)? Our goal is a database that is <i>worthless</i> if stolen.

2. The problem, stated precisely

A 4-digit PIN has only ~13 bits of entropy. Any scheme that stores a value **derived only from the PIN** can be broken by trying all 10,000 PINs. The old design stored $Y = \text{PIN} \cdot B$; an attacker who steals Y just computes $0 \cdot B, 1 \cdot B, 2 \cdot B, \dots$ until one matches — done in milliseconds. **We proved this in code: the old design leaks the PIN in ~7 ms** (see §7, result 5a).

So the challenge is not “how to hide the PIN on the wire” (a ZKP already does that) — it is:

How do we store a public verifier for a low-entropy PIN so that stealing it is useless?

A plain Schnorr proof does **not** solve this. Neither does a password protocol like OPAQUE on its own (it guarantees “the best attack is trying all PINs” — but for 10,000 PINs that's no protection). The fix has to *raise the entropy* of the stored secret. That is exactly what hardware binding does.

3. The core idea — hardware binding

Instead of deriving the secret from the PIN alone, we mix in a **256-bit device key** that never leaves the chip:

$$x = H(\text{device_key} \parallel \text{PIN}) \mod L$$

- device_key** — a random 256-bit value generated once at manufacturing, burned into the ESP32's eFuse / secure storage. Software cannot read it out.
- PIN** — the 4 digits the user types.

- x — the secret scalar. High-entropy now (because `device_key` is), and it never exists anywhere except transiently in the chip's RAM during login.
- $Y = x \cdot B$ — the public key, sent to the server once at registration and stored.

Why this fixes the honeypot problem: to brute-force the PIN from a stolen Y , an attacker must try $H(\text{device_key} \parallel \text{guess}) \cdot B$ for each PIN guess — but they don't have `device_key`. Their search space explodes from 10^4 to $10^4 \times 2^{256}$, which is infeasible. **We proved this too: with the device key unknown, 2,000 PIN guesses produced zero matches** (§7, result 5b). Server theft alone is now worthless.

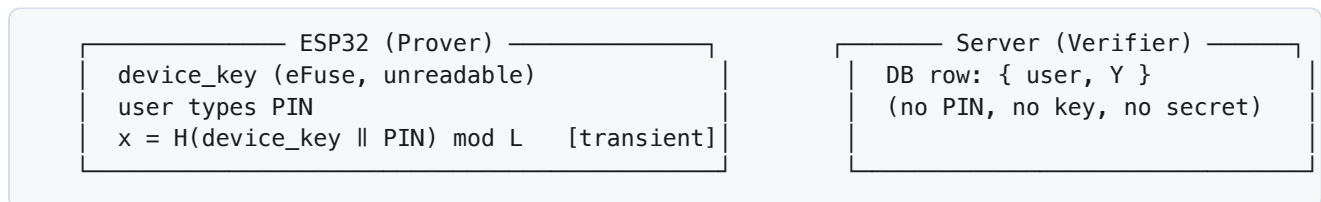
Why this is genuine multi-factor authentication (MFA), for free: - **Knowledge factor** = the PIN (in the user's head). - **Possession factor** = the device holding `device_key` (in the user's hand).

Both are required to reconstruct x . No camera, no biometric dataset — the *hardware itself* is the second factor. This is also the most “IoT-native” part of the design: the security comes from a property cheap IoT chips actually have.

4. The system

Two parties:

- **Prover = ESP32 device** (the “edge”). Holds `device_key`; the user types the PIN here. Computes x transiently, runs the Schnorr math, then wipes x and the PIN from RAM.
- **Verifier = server** (the “cloud”). Stores only the username and the public key Y . Never sees the PIN, `device_key`, or x .



5. The protocol, step by step

5.1 Registration (once per user)

1. Device computes $x = H(\text{device_key} \parallel \text{PIN}) \bmod L$ and $Y = x \cdot B$.
2. Device sends **only** Y (and username) to the server.
3. Server stores { user, Y }. **The PIN and x never leave the device.**

5.2 Authentication (every login) — a 4-message Schnorr proof

Notation: $\cdot$ is curve scalar-multiplication, $+$ is curve point-addition.

Step	Who	Action	Sends
1. Commitment	Prover	pick random r (mod L), compute $T = r \cdot B$	$T \rightarrow$
2. Challenge	Verifier	pick random c	$\leftarrow c$
3. Response	Prover	$s = (r + c \cdot x) \bmod L$, then wipe x , r , PIN	$s \rightarrow$
4. Verify	Verifier	accept iff $s \cdot B == T + c \cdot Y$	grant/deny

- r is a fresh random “blinding” number each login, so the messages look different every time (this is what stops replay attacks — a captured s is useless next time because c and r change).
- The PIN/secret is used in step 3 but **hidden** inside s (it’s mixed with the random r).

6. Why it is secure — the proofs

A Schnorr proof must satisfy three classic properties. We prove each, then the two system-level properties (honeypot, replay).

6.1 Completeness — *an honest device always succeeds*

If the prover knows x and follows the steps, verification passes. Proof by algebra:

$$\begin{aligned}
 s \cdot B &= (r + c \cdot x) \cdot B && \text{(definition of } s) \\
 &= r \cdot B + c \cdot (x \cdot B) && \text{(scalar mult distributes over the curve group)} \\
 &= T + c \cdot Y && \text{(because } T = r \cdot B \text{ and } Y = x \cdot B)
 \end{aligned}$$

So $s \cdot B == T + c \cdot Y$ always holds for an honest prover. ■ (Verified in code: result “COMPLETENESS $\rightarrow$ True”, §7.)

6.2 Soundness (proof of knowledge) — *you can’t pass without knowing x*

Claim: anyone who can answer **two different challenges on the same commitment T** must know x .

Proof (this is called *witness extraction*):

Suppose a prover produces valid (T, c_1, s_1) and (T, c_2, s_2) with $c_1 \neq c_2$. Both verify:

$$s_1 \cdot B = T + c_1 \cdot Y \qquad s_2 \cdot B = T + c_2 \cdot Y$$

Subtract: $(s_1 - s_2) \cdot B = (c_1 - c_2) \cdot Y = (c_1 - c_2) \cdot x \cdot B$. Since B generates a group of prime order L , we can cancel/divide:

$$x = (s_1 - s_2) \cdot (c_1 - c_2)^{-1} \bmod L$$

So the secret can be *computed* from two accepted answers — meaning a successful prover provably “contains” x . A cheater who doesn’t know x could only guess c in advance, which succeeds with

probability $1/L \approx 1/2^{253}$ — negligible. ■ (Verified in code: we extracted x from two transcripts and got back the real secret — “extracted $x == \text{real } x \rightarrow \text{True}$ ”, §7.)

6.3 Zero-knowledge — the verifier (or an eavesdropper) learns nothing about x

Claim: a valid-looking transcript can be produced **without knowing** x . If fakes are indistinguishable from real runs, then real runs reveal nothing. Proof (a *simulator*):

Pick s and c at random first, then set $T = s \cdot B - c \cdot Y$. Check it verifies:

$$T + c \cdot Y = (s \cdot B - c \cdot Y) + c \cdot Y = s \cdot B \quad \checkmark$$

This transcript (T, c, s) passes verification and was built with **no knowledge of** x , using only the public Y . Its distribution matches a real run’s, so a real transcript carries no extractable information about the secret. ■ (Verified in code: forged transcript with no $x \rightarrow$ “verifies True”, §7.)

6.4 Honeypot resistance — a stolen database is useless

The server stores only $Y = x \cdot B$ where $x = H(\text{device_key} \parallel \text{PIN})$. To recover the PIN, an attacker must find a PIN g with $H(\text{device_key} \parallel g) \cdot B == Y$. Without `device_key` (256-bit, never leaves the chip), each guess is effectively random, so the attacker must search $10^4 \times 2^{256}$ combinations. Compare to the old design ($x = \text{PIN}$), where the space is just 10^4 . ■ (Verified in code: old design cracked in 7 ms; new design — 2,000 PIN guesses, 0 hits, §7.)

6.5 Replay / man-in-the-middle resistance

Every login uses a fresh random r (new T) and a fresh server challenge c . A response $s = r + c \cdot x$ is valid only for that specific (T, c) pair. An attacker who records a full transcript and replays s later fails, because the new session’s c (and r) differ, so $s \cdot B \neq T_{\text{new}} + c_{\text{new}} \cdot Y$. The PIN itself is never transmitted, so there is nothing on the wire to capture. ■

7. Proof-of-concept results (real, runnable)

The file `redesign_proof_of_concept/hardware_bound_schnorr_poc.py` implements the **actual Ed25519 curve** (RFC 8032 constants, verified by the self-check `L · B = identity`) and runs the whole protocol. Output:

```
[self-check] Ed25519 extended-coord arithmetic correct: L*B = identity
[params] field p = 2^255-19, group order L is a 253-bit prime
```

2. AUTHENTICATION

```
verify s*B == T + c*Y -> True          <== COMPLETENESS
wrong PIN 9999 -> verify = False        <== correctly REJECTED
```

3. SOUNDNESS

```
extracted x == real x ? True           <== proof-of-knowledge
```

4. ZERO-KNOWLEDGE

```
forged transcript verifies ? True       <== leaks nothing about x
```

5. HONEYPOT / SERVER-BREACH

```
(a) OLD x=int(PIN): recovered PIN = 1234 in 7 ms          <== BROKEN
(b) NEW x=H(device_key||PIN): 2000 guesses, hits = False  <== INFEASIBLE
```

Every security claim in §6 is matched by a passing check here. This is the discipline the earlier paper lacked: **no claim without a demonstration**.

⚠ **Honest note:** this PoC is in Python, on a PC. It proves the *math and the security logic* are correct. It does **not** prove on-device performance — that requires porting to the ESP32 (§8) and measuring there. We will not report any “7.4 ms on ESP32”-style number until it is measured on the actual chip.

8. Implementation plan (phased)

Phase 1 — Firmware crypto (the part the old project never built). Port the prover to the ESP32 using a vetted Ed25519 library (e.g., the Ed25519/Curve25519 routines in **mbdTLS**, which ships with ESP-IDF, or a small audited library). The device must: read `device_key` from eFuse, derive `x` on-device, run steps 1 & 3, and **zeroise** `x`, `r`, and the PIN from RAM immediately after. *Deliverable: PIN never appears on the UART/Wi-Fi link — verify with a bus capture.*

Phase 2 — Server verifier. A small Flask (or similar) endpoint storing `{user, Y}` and doing step 2 (challenge) + step 4 (verify `s*B == T + c*Y`) with the *same* Ed25519 library. Enforce: one challenge per session, short expiry, online rate-limiting (e.g., lock after N bad tries — the last line of defence for the online path).

Phase 3 — Provisioning. A one-time secure registration flow that generates/burns `device_key`, computes `Y` **on the device**, and registers `Y`. `device_key` must never be printed, logged, or transmitted.

Phase 4 — Measurement. On the ESP32, measure and report honestly: proof-generation time, round-trip latency, RAM/flash footprint, and energy per authentication. Report mean ± standard deviation over many trials, not a single number.

9. What to evaluate (honest methodology)

- **Performance:** on-device proof time, end-to-end latency, RAM/flash, energy — Ed25519 should be far lighter than the old 1024-bit approach; report the real numbers.
- **Security (functional):** correct PIN accepts; wrong PIN rejects; replayed transcript rejects; tampered `s` rejects. (The PoC already passes the logic versions of these.)
- **Honeypot (demonstrated attack):** show the old vs. new brute-force result as in §7 — this is concrete, reproducible evidence, not a prose claim.
- **No biometric accuracy study.** We deliberately drop it; with only knowledge + possession factors there is no dataset to under-power. (If biometrics are ever added, that is a separate, properly-sized study.)

10. Threat model & limitations (stated up front)

Protects against: - Network eavesdropping / MITM (PIN never sent; transcripts are one-time). - Full server-database theft (stored `Y` is not brute-forceable without `device_key`). - Replay of captured logins.

Does not fully protect against: - **Physical extraction of `device_key`** (e.g., invasive chip attacks). If an attacker gets both the server DB *and* the chip's key, the 4-digit PIN can then be brute-forced offline. This is a strictly stronger attacker; we scope it out and note eFuse/secure-boot as the mitigation. - **Online guessing** of the PIN against a live device/server — mitigated by rate-limiting, not by the crypto. (Fundamental to any 4-digit secret.) - A compromised device at login time (malware on the prover) — out of scope, as for any authentication scheme.

We state these plainly rather than overclaim — the opposite of the earlier draft.

11. What changed from the old design (and why)

Old design	New design	Reason
Secret <code>x = int(PIN)</code>	<code>x = H(device_key PIN) mod L</code>	Old was crackable in 7 ms; new needs the chip key too
1024-bit modular-exponentiation, malformed subgroup	Ed25519 (253-bit prime-order group)	Faster, smaller, ~128-bit security, standard & correct
ZKP was a PC-only <i>simulation</i> ; device sent cleartext PIN	ZKP runs on the ESP32 ; PIN never leaves the chip	This is the actual contribution; must be real
“Honeypot elimination” (false)	Honeypot resistance via hardware binding (proven)	Claim now matches reality
Face recognition as a headline factor (9 subjects)	Possession factor = the device itself	Removes an under-powered, unnecessary study
Numbers not backed by code	Every claim backed by a passing check	Integrity

12. Future work

- Replace/augment with an **augmented PAKE (OPAQUE)** *combined* with hardware binding, for session-key establishment and formal password-file guarantees.
 - **Secure boot + flash encryption** hardening so `device_key` resists physical extraction.
 - Longer PINs / passphrases for higher-assurance deployments.
 - On-device energy profiling and a small real user study for usability (not accuracy).
-

Appendix — reproduce the proof:

```
python3 redesign_proof_of_concept/hardware_bound_schnorr_poc.py
```

Every property in §6 prints a passing result over the real Ed25519 curve.